

AIR UNIVERSITY
Department of
Mechatronics
College of Engineering

ARTIFICIAL INTELLIGENCE LAB

LAB 08

Lab Objective: Perceptron Learning Algorithm

Class ID: _____

LabDate: _____

Student's ID: _____

LAB 08		Perceptron Learning Algorithm			
PLO's	PLO3 – Design and Development		Bloom's Taxonomy	C3 – Apply	
LAB TASK PERFORMANCE					
CLO's	Aspects of Assessments	Excellent (75-100%)	Average (50-75%)	Poor (<50%)	M
CLO2 100%	<u>Design & Development</u> Design solution for problem that meets specified needs by using the concept of artificial intelligence.	A complete solution / Explain necessary requirements according to task description with great use of time and resource material.	Solution was complete but need minor modifications /student could have followed specification more closely.	Solution was complete but did not work, needed several modifications / did not make correct use of resource material or instructions.	
Total Marks: 10					

Graded by Engineer: _____

Date: _____

Remarks: _____

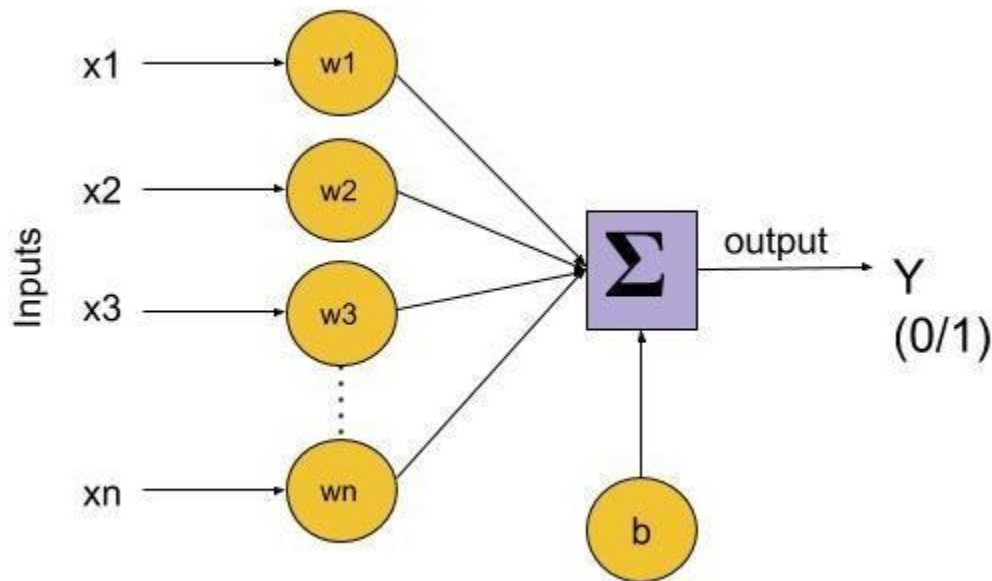
Objective:

1. Make attendee familiar with the concept of perceptron
2. Make attendee understand perceptron learning algorithm and the relevant weight update rules
3. Hands-on activity

Perceptron

The idea of a Perceptron is analogous to the operating principle of the basic processing unit of the brain — Neuron. A Neuron is comprised of many input signals carried by Dendrites, the cell body and one output signal carried along Axon. The Neuron fires an action signal when the cell meets a particular threshold. This action either happen or they don't; there is no such thing as a “partial” firing of a neuron.

Similarly, the perceptron has many inputs(often called features) that are fed into a Linear unit that produces one binary output. Therefore, perceptrons can be applied in solving Binary Classification problems where the sample is to be identified as belonging to one of the predefined two classes.

The Perceptron Learning Algorithm

Since Perceptrons are Binary Classifiers (0/1), we can define their computation as follows:

$$f_{\Phi}(\mathbf{x}) = \begin{cases} 1 & \text{if } b + \mathbf{w} \cdot \mathbf{x} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Let's recall that the dot product of two vectors of length n ($1 \leq i \leq n$) is $\mathbf{w} \cdot \mathbf{x} = \sum_i w_i \cdot x_i$. The function $f(\mathbf{x}) = b + \mathbf{w} \cdot \mathbf{x}$ is a linear combination of weight and feature vectors. Perceptron is, therefore, a linear classifier — an algorithm that predicts using a linear predictor function.

The weights signify the effectiveness of each feature x_i in $\mathbf{x}$ on the model's behavior. Higher the weight w_i of a feature x_i , higher is its influence on the output. On the other hand, the bias 'b' is like the intercept in the linear equation. It's a constant that helps the model adjust in a way that best fits the data. The bias term assumes an imaginary input feature coefficient $x_0=1$.

The model can be trained using the following algorithm:

1. set $b = w = 0$
2. for N iterations, or until weights do not change
 - (a) for each training example x^k with label y^k
 - i. if $y^k - f(x^k) = 0$, continue
 - ii. else, update w_i , $\Delta w_i = (y^k - f(x^k)) x_i$

Implementation

The dataset that we consider for implementing Perceptron is the [Iris flower dataset](#). This dataset contains 4 features that describe the flower and classify them as belonging to one of the 3 classes. We strip the last 50 rows of the dataset that belongs to the class 'Iris-virginica' and use only 2 classes 'Iris-setosa' and 'Iris-versicolor' because these classes are linearly separable and the algorithm converges to a local minimum by eventually finding the optimal weights.

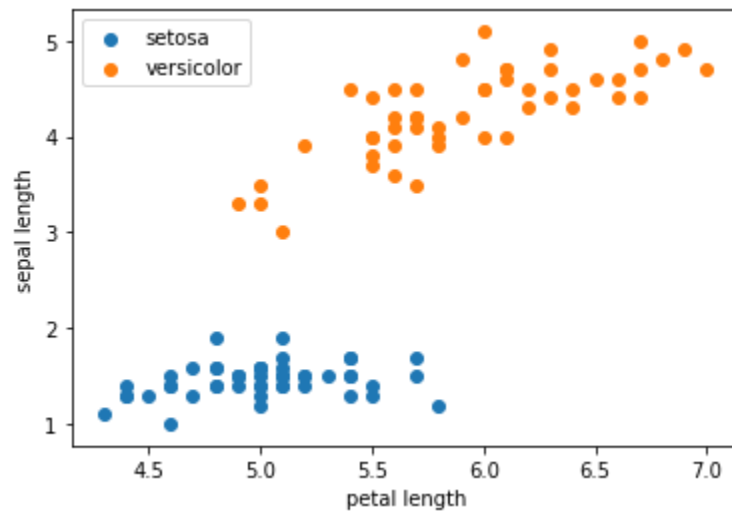
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
def load_data():
    URL_='https://archive.ics.uci.edu/ml/machine-learning-
databases/iris/iris.data'
    data = pd.read_csv(URL_, header = None)
    print(data)

    # make the dataset linearly separable by selecting samples from two classes
    only
    data = data[:100]
    data[4] = np.where(data.iloc[:, -1]=='Iris-setosa', 0, 1)
    data = np.asmatrix(data, dtype = 'float64')
    return data
data = load_data()
```

Visualizing the dataset with 2 of the features, we can see that that dataset can be clearly separated by drawing a straight line between them.

Our goal is to write an algorithm that finds that line and classifies all of these data points correctly.

```
plt.scatter(np.array(data[:50,0]), np.array(data[:50,2]), marker='o', label='setosa')
plt.scatter(np.array(data[50:,0]), np.array(data[50:,2]), marker='x', label='versicolor')
plt.xlabel('petal length')
plt.ylabel('sepal length')
plt.legend()
plt.show()
```



Now we implement the algorithm mentioned above as it is and see how it works. We have 4 features and hence 4 weights associated with each feature. Remember that we defined a bias term w_0 that assumes $x_0=1$ making it a total of 5 weights.

We have defined the number of iterations to be 10. This is one of the *hyperparameters*, as opposed to system parameters like w that are learned by the algorithm. At each iteration, the algorithm computes the class (0 or 1) for all the data points and updates the weights with each misclassification.

If the sample is misclassified, then the weights are updated by Δ that shifts in the opposite direction. So if the sample is to be classified again, the result is “less wrong”. We classify any $\text{label} \leq 0$ as ‘0’ (Iris-setosa) anything else to be a ‘1’ (Iris-versicolor)

```
def perceptron(data, num_iter):
    features = data[:, :-1] ##take all data except the last column
    labels = data[:, -1] ##take only the last column

    # set weights to zero
    w = np.zeros(shape=(1, features.shape[1]+1))
```

```

misclassified_ = [] ##Used by convention to avoid naming conflicts with P
ython keywords.

for epoch in range(num_iter):
    misclassified = 0
    for x, label in zip(features, labels): ##The zip() function returns a
zip object, which is an iterator of tuples
        #where the first item in each passed iterator is paired together, and
then the second item in each passed iterator are paired together etc
        x = np.insert(x,0,1) ## insert 1 to make DC term work by fixing
an artificially generated feature value as 1
        y = np.dot(w, x.transpose())
        target = 1.0 if (y > 0) else 0.0 ##hardlimiter

        delta = (label.item(0,0) - target) ## actual class - predicted
class

        if(delta): # misclassified
            misclassified += 1
            w += (delta * x)

    misclassified_.append(misclassified)
return (w, misclassified_)

num_iter = 10
w, misclassified_ = perceptron(data, num_iter)
misclassified_

```

Lab Task:

(CLO4, Marks 4) Complete hands-on activities as prescribed above. **Analyze** the change in behavior of the classification accuracy of the perceptron algorithm (**through misclassified cases**) as the number of iterations are progressively varied from 1 to 10 in steps of 1. Plot graph and make a table for the results.

Analyze the change accuracy of the algorithm if the number of features are reduced from 4 to 3 to 2. **Comment** on it explaining the change.

(CLO 6, Marks 1) Perform properly handling of lab.

Further Lab Tasks:

4. (CLO4, Marks 5) **Solve** the Boolean OR problem using the Perceptron Learning Algorithm by **designing** a suitable perceptron algorithm. Once the learning is complete, perform testing on all four samples and show that misclassification is zero. Submit your python code as well.